

What I'd Ship Next — One More Week

This roadmap was written after I ran the system against my own evaluation framework and then audited it from a Staff Engineer's perspective — looking for the gaps that separate a working demo from a production system. The five items below are ordered by architectural severity, not by implementation ease. Each one addresses a real weakness I surfaced during testing, documented with the mitigation I'd ship.

1. INGEST-TIME COMPRESSION TO PRESERVE PROMPT CACHE (DAYS 1-2)

The problem. The current architecture runs compression at query time — every new query recomputes the compressed history. This is architecturally incompatible with Anthropic's prompt caching, which rewards stable prefixes. Any modification to the middle of the conversation invalidates the cache on everything after the modification. The practical impact: a production deployment using this system as-is sees ~0% cache hit rate on the compressed portion, which means the true economic savings are meaningfully worse than my eval reports. My eval measured input-token cost; it did not model the cache-eligible baseline a real production system would have.

The fix. Move from query-time to ingest-time compression. Compress old messages once — when they pass an age or length threshold — and freeze them. New turns append to a stable prefix. The cache hits everything above the last compression boundary. Re-compression only happens at "chapter boundaries" (e.g. every 50 messages), not on every query. The router decides when a chapter closes.

Why it matters. This is the difference between a system that looks good in an eval and one that actually saves money in production. Without this, a CFO does the math and finds the savings smaller than claimed. With it, the system is cache-friendly and the net positive is real.

Implementation. Add a `ConversationChapter` abstraction — a frozen span of compressed history with a content hash. Store chapters keyed by prefix hash. Optimizer reads chapters sequentially, only re-processes the live tail. Two weeks' eng work to do properly; one week to ship a prototype.

2. SYMMETRIC TOOL-CALL PAIR VALIDATION + RECENCY SAFETY BUFFER (DAYS 2-3)

The problem — two related data-integrity gaps I found while auditing the assembler.

First, the assembler handles orphaned `tool_result` (skip it) but not orphaned `tool_use` (keeps it, which breaks the chain). If compression discards a tool result that was scored as noise, the surviving tool call has no output. Sonnet either hallucinates an answer or errors. My eval didn't catch this because my synthetic conversations use no tools.

Second, there is no hard safety buffer protecting recent messages. The recency decay weight biases toward keeping them (0.10-0.45 depending on query profile) but nothing guarantees it. If a recent commitment is phrased informally ("yeah £40 works for me") and the regex misses it, it can be classified as noise and dropped.

The fix. Two small changes, both low-risk. First: bidirectional pair validation in `_validate_and_repair` — if a `tool_use` survives but its `tool_result` doesn't, restore the result from the pre-compression buffer or drop the call. Second: a last-K floor in the compressor where K is tunable per query profile (FACTUAL: K=10; STATUS: K=15; ANALYTICAL: K=5). Recent messages below the floor are always kept, regardless of score.

Why it matters. Both failures are silent. The system returns a "clean" optimized context that is actually corrupt (broken tool chain) or incomplete (missing recent commitment). A missed commitment in a trade negotiation is not a quality regression — it is a commercial liability. For LEC specifically, this is the gap that matters most.

Implementation. ~30 lines in `assembler.py` for the pair check. ~20 lines in `compressor.py` for the safety buffer. Add stress tests: orphan-tool-use, informal-commitment, multilingual-landmark.

3. CROSS-FAMILY LLM-AS-JUDGE TO NEUTRALIZE HOMOPHILY (DAY 3)

The problem. My current eval uses Claude Haiku to compress AND Claude Haiku to judge — same model family on both sides. This is a known bias vector: judges favour output that matches their own stylistic priors. The corroborating signal in my results is ceiling saturation — every query scored 5.0/9 on both full and optimised answers, producing a delta of exactly zero. That pattern is consistent with a judge being generous across the board because the answers look "right" in its own style.

The fix. Add a second judge — GPT-4o or Gemini 2.5 Pro — cross-family to Claude. Score every eval case with both judges. Use the minimum score per case (adversarial aggregation) or report both separately and flag divergence. Also redesign the rubric to include partial-answer traps that force judge separation — queries where a plausible wrong answer is available in the compressed context, so a lenient judge can't default to max score.

Why it matters. The requirement in the brief — "optimised context performs $\geq$ full context on the quality metric" — is only meaningful if the quality metric discriminates. My current rubric doesn't. A senior reviewer will notice the 5.0/9 saturation and ask how I know my system actually works versus just not failing obviously. Cross-family judging is the answer.

Implementation. One-file change in `eval_runner.py`. OpenAI API client added to requirements. Re-run eval, produce delta charts. Two days of work to do properly including rubric redesign.

4. HIERARCHICAL MEMORY FOR 500+ MESSAGE CONVERSATIONS (DAYS 3-5)

The problem. At current scale (51 messages), semantic scoring runs in <300ms cold-cache. At 500 messages, computing cosine similarity across every message in one batch hits a latency wall — roughly 7-12s on CPU before compression even starts. This is one of the three hard-mode signals the brief explicitly asks about, and honestly addressing it matters.

The deeper issue: even if embedding were free, single-pass compression can't handle conversations at 1000+ messages. You need to summarize summaries. That introduces "Digital Alzheimer's" — semantic drift after 3-4 recursive passes where the original intent is lost. My current system doesn't do recursive summarization at all, which means it has a hard practical ceiling around 500 messages.

The fix. Hierarchical memory with Qdrant as the store. Chunk the conversation into 50-message windows, summarize each window with landmarks preserved. Store summaries as semantic nodes in Qdrant. At query time: retrieve the top-K relevant chunks from the vector store, hydrate their landmarks verbatim, pass a tight context forward. Cap recursion at 2 levels to bound drift — deeper than that, landmark-anchor-only retrieval.

Why it matters. LEC runs long-cycle conversations. A trade negotiation or a due-diligence thread can run hundreds of turns over weeks. The system needs to scale past my current tested range. Without this, I've built something that works at 50 messages and guessed at 500.

Implementation. `qdrant-client` SDK, Docker for local dev, Qdrant Cloud for production. Add `ConversationStore` as a persistence layer above the optimizer. Add a length-threshold router — under 100 messages, skip the store entirely. Week of work; one day for a minimum prototype that demonstrates the pattern at 200 messages.

5. OBSERVABILITY + LENGTH-THRESHOLD ROUTER (DAYS 5-6)

The problem. Two problems, one solution. First: my net cost analysis is currently negative on short conversations (51 messages, mostly factual queries). Compression overhead beats token savings at that scale. Shipping the current system to all conversations loses money. Second: once the other fixes are in place, there's no visibility into whether they're actually delivering — which conversations compress well, which query types have the lowest win rates, whether win rate regresses after a prompt change.

The fix. Two thin layers. First: a router that decides whether to invoke the optimizer at all, based on conversation length, landmark density, and historical win rate for that conversation type. Below threshold, pass full context through untouched. Above threshold, route to optimizer. Turns the system from a blunt instrument into a selective one.

Second: OpenTelemetry instrumentation on every pipeline stage, exported to Grafana. Dashboards for token reduction distribution, win rate by query type, compression cost vs savings, p50/p95 latency, and projected monthly saving. Add LLM-as-judge score drift tracking to catch quality regressions within hours.

Why it matters. The router alone turns the system from a "maybe net positive" to a "provably net positive" deployment. Observability is what lets a VP of Engineering see ROI in real time and what lets me diagnose regressions before they become outages. Together they're what makes the difference between "we built a thing" and "we run a service."

Implementation. Router: ~100 lines in a new `optimizer/router.py`, config-driven thresholds, rule-based to start, ML-learnt later. Observability: `opentelemetry-sdk` + Prometheus exporter, Grafana dashboard committed to the repo. Two days total if the groundwork from items 1-4 is in place.

PRIORITY REASONING

The ordering above prioritises **architectural correctness over feature breadth**. Item 1 (ingest-time compression) goes first because it invalidates my own cost claims until fixed. Item 2 (integrity fixes) goes second because silent data corruption in a trade-negotiation context is a commercial risk, not just a quality regression. Item 3 (judge bias) goes third because it gates the credibility of every eval number I report. Items 4 and 5 extend the system to production scale once the foundations hold.

What I'm explicitly deferring past week one: recursive summarisation beyond 2 levels (needs a research pass on drift thresholds), fine-tuning a small model for compression (eliminates API dependency but costs weeks), and multilingual landmark detection (needs a zero-shot classifier and a multilingual eval dataset). All three matter. None are the bottleneck right now.